71-FOC 24

大家晚上好，欢迎参加张飞实战电子 FOC 第 11 期学习。上节课我说要准备讲一下自适应滤波系数的求取。y(n) = y(n-1) + K(x(n) - y(n-1))，这里的 K 是个系数。

$$factor = Ts * 2\pi * fc$$

大家看，这是我们使用的一阶数字低通滤波器系数的计算公式，对吧？ 系数中，Ts 跟 2π 都相当于是已知的量。我们要确定滤波系数的话，重点是 fc 截止频率的确定。而我们之前也说过，对控制器的输出滤波，要想有固定的相位延迟，是需要让截止频率跟电频率一致所以，确定截止频率的话，也就是我们要求解出电频率。

$$\omega = 2\pi * f$$

那么，这里我们知道，电频率 f 跟角频率 ω 的关系。然后呢，我们又知道角频率跟角度变化量的关系。

$$\omega = \frac{\Delta\theta}{\Delta t}$$

当然，因为表示的是角频率，所以这个式子中的角度变化量 Δθ 的单位应该是用弧度表示的。然后，结合上面两个式子，就可以得到：

$$f = \frac{\frac{\Delta\theta}{\Delta t}}{2\pi}$$

这样的话，也就得到电频率的求解公式。

$$f = \frac{\Delta\theta}{2\pi} * \frac{1}{\Delta t}$$

或者呢，我们可以写为这样。那么，这里的 f 的求取的话，也就是把角度变化量求取出来。

问：还有个 C 呢？

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

答：fc 的 C 是个下标，fc 是截止频率的意思，英文全称应该是 cut， fcut。

$$factor = Ts * 2\pi * \frac{\Delta\theta}{2\pi} * \frac{1}{\Delta t}$$

所以，这个低通滤波系数的话，我们就可以写为这样。而因为我们使用反正切求解出来的角度是 $-\pi \sim +\pi$ 表示的。所以呢，上面这个式子，我们可以再稍微变化一下。

$$factor = \frac{Ts * \pi}{\Delta t} * \frac{\Delta\theta}{\pi}$$

例如呢，我们可以变为这样。

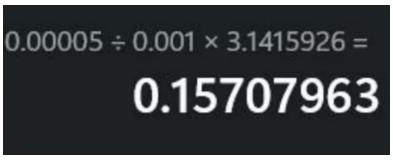
$$factor = \frac{Ts * \pi}{\Delta t} * \boxed{\frac{\Delta\theta}{\pi}}$$

那么，这个蓝框的部分，正好是角度的归一化。这个部分归一化之后，就是-1~1 之间的量。而我们软件编程的时候呢，是把-1~1 之间的数转换为 Q15 格式了。所以，这个蓝框的部分，在我们软件中呢，Q15 的范围就是 -32768~32767。

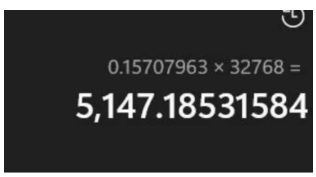
我们继续往下讲哈。

$$factor = \boxed{\frac{Ts * \pi}{\Delta t}} * \boxed{\frac{\Delta\theta}{\pi}}$$

我们程序中计算这个滤波系数的话，是分成了两个部分相乘实现的。Ts = 50us，Δt = 1ms π = 3.1415926。所以呢，蓝框部分就相当于都是已知的量，可以用宏定义。

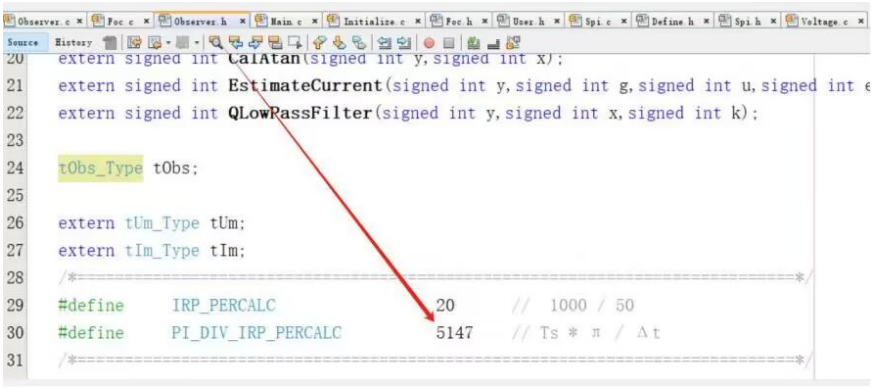


计算出来呢，这个部分对应的小数是 0.15707963，然后，编程的时候，需要转换为 Q15，因此呢，需要计算 0.15707963 * 32768 的值。



计算出来呢，是这个，我们也就取 5147 即可。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



这是我们程序中定义的这个宏定义。

$$factor = \frac{Ts * \pi}{\Delta t} * \frac{\Delta \theta}{\pi}$$

接下来，我们来看下这个蓝框部分的代码实现。Δ θ 的话，是单位时间的角度变化量单位时间，我们取的是 1ms。

```
tObs.qThetaSum += qThetaTemp - tObs.qThetaLast;
tObs.qThetaLast = qThetaTemp;

tObs.qThetaSumCnt ++;
if (tObs.qThetaSumCnt >= IRP_PERCALC)        //1000us / 50us
{
    tObs.qOmega = tObs.qThetaSum;
    tObs.qThetaSumCnt = 0;
    tObs.qThetaSum = 0;
}
```

所以呢，我们这里就是每 50us 计算一次角度差，并累加。当 1ms 到了之后，就取累加的值，也就得到了 1ms 内的角度变化量。

问：角度有负值吗？累加的话会不会超？

答：不会，你可以计算一下，如果超过的话，速度是多少。这个时间的选取会确保转速在设计范围内。之前好像算过了，目前这个取值可以至 3 万转。

而这里呢，这个值已经是归一化之后，处理为 Q15 的角度了。所以，这个 qOmgea 相当于 Δ θ / π * 32768。那么，理论上，求取滤波系数，我们取这个 tObs.qOmega 去跟 刚才计算出来的 5147 相乘即可。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
tObs.qThetaSumCnt ++;
if (tObs.qThetaSumCnt >= IRP_PERCALC)        //1000us / 50us
{
    tObs.qOmega = tObs.qThetaSum;
    tObs.qThetaSumCnt = 0;
    tObs.qThetaSum = 0;
}
tObs.qKomega = KOMEGAFILTED;
tObs.qOmegaFilted = QLowPassFilter(tObs.qOmegaFilted, tObs.qOmega, tObs.qKomega);

sTemp = tObs.qOmegaFilted;
if (sTemp < OMEGA300RPM)
    sTemp = OMEGA300RPM;
tObs.qKafc = tObs.qKafcFinal = MulFrac(sTemp, PI_DIV_IRP_PERCALC);
tObs.qTheta = qThetaTemp + EANGLE90;
```

大家看，假设我们暂且忽略掉中间的滤波操作。

$$factor = \frac{Ts * \pi}{\Delta t} * \frac{\Delta \theta}{\pi}$$

那么，箭头中所指的地方，就是计算蓝框的乘积。好，关于软件实现自适应低通滤波系数的这个部分。

那我们继续啊。

```
tObs.qThetaSumCnt ++;
if (tObs.qThetaSumCnt >= IRP_PERCALC)        //1000us / 50us
{
    tObs.qOmega = tObs.qThetaSum;
    tObs.qThetaSumCnt = 0;
    tObs.qThetaSum = 0;
}
tObs.qKomega = KOMEGAFILTED;
tObs.qOmegaFilted = QLowPassFilter(tObs.qOmegaFilted, tObs.qOmega, tObs.qKomega);

sTemp = tObs.qOmegaFilted;
if (sTemp < OMEGA300RPM)
    sTemp = OMEGA300RPM;
tObs.qKafc = tObs.qKafcFinal = MulFrac(sTemp, PI_DIV_IRP_PERCALC);
```

而我们实际在使用的时候呢，对单位时间内的角度变化量做了一个低通滤波。因为这个值变化也不大，但是它稳定一些的话，后面计算的角度也就会平滑一些。所以，就做了低通滤波，这里低通滤波系数，我们取的是电机工作最低转速，这里取 300RPM（机械转速）对应的电频率来作为截止频率来滤波的。所以，我们需要把 300RPM 对应的电频率计算出来，因为我们使用的电机，极对数是 2 对极。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

$$RPM = \frac{f * 60}{P}$$

转速跟电频率之间，有这样的关系。这里极对数 P = 2， RPM = 300。

$$f = \frac{300 * 2}{60} = 10$$

这样的话，就可以把 f 给计算出来。

$$f = \frac{\Delta\theta}{2\pi} * \frac{1}{\Delta t}$$

那么，f 的公式是这个。最终，可以把 Δθ/π 计算出来。

$$10 = \frac{\Delta\theta}{2\pi} * \frac{1}{\Delta t}$$

$$\Delta\theta = 0.062831852$$

$$\frac{\Delta\theta}{\pi} = 0.02$$

$$factor = \frac{Ts * \pi}{\Delta t} * \frac{\Delta\theta}{\pi}$$

而这里 Ts*π/Δt = 0.15707963，所以，此时的滤波系数就是 0.15707963 * 0.02 = 0.0031415926。然后，把这个 0.0031415926 转换为 Q15，也就是 103。

```
/*=============================================================
#define     OMEGA300RPM                 655     //
/*=============================================================
#define     KOMEGAFILTED                103     //
```

这个值呢，也使用宏定义定义出来了。

```
}
tObs.qKomega = KOMEGAFILTED;
tObs.qOmegaFilted = QLowPassFilter(tObs.qOmegaFilted, tObs.qOmega, tObs.qKomega);
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

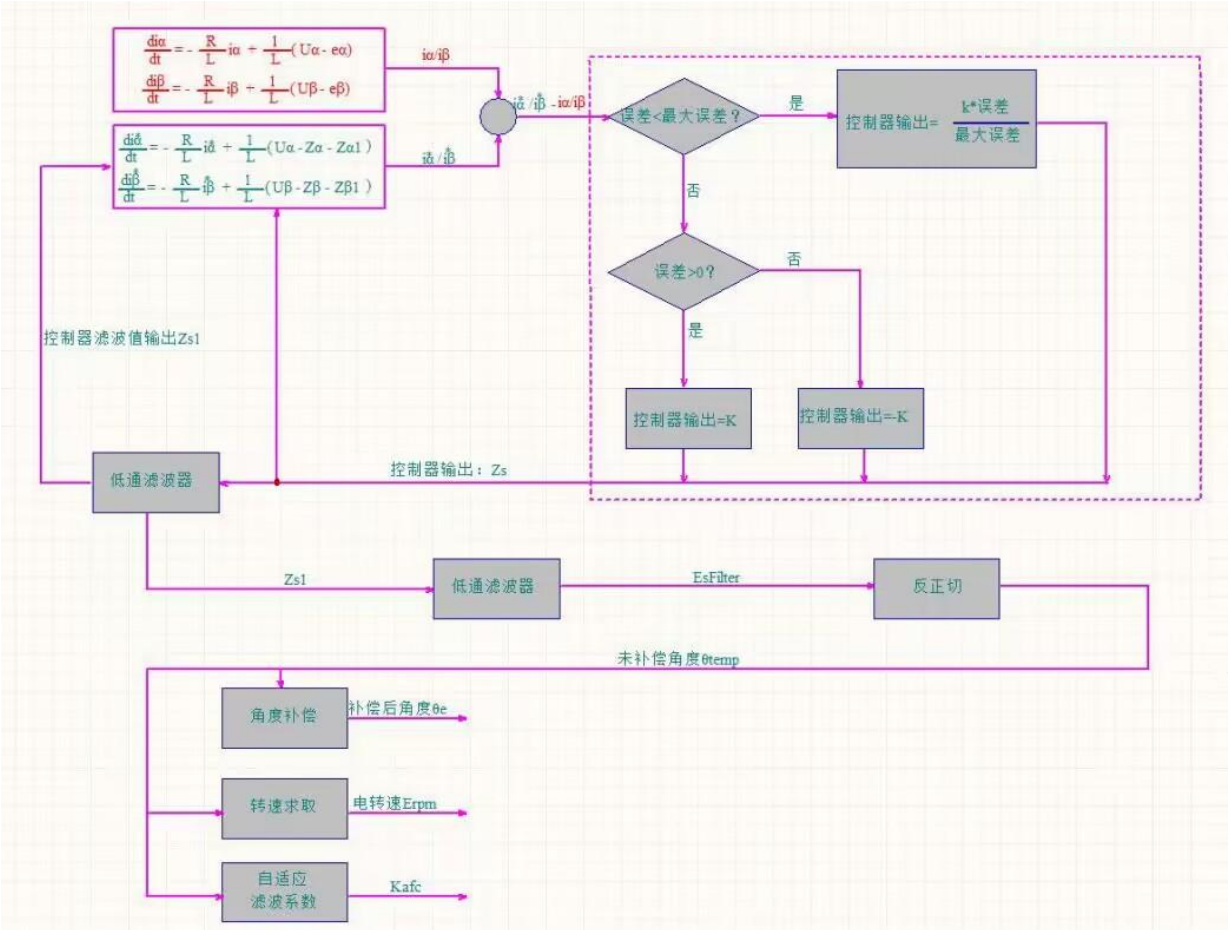
然后，调用我们之前编写出来的低通滤波函数，就可以对 Omega 滤波了。

```
tObs.qThetaSumCnt ++;
if (tObs.qThetaSumCnt >= IRP_PERCALC)        //1000us / 50us
{
    tObs.qOmega = tObs.qThetaSum;
    tObs.qThetaSumCnt = 0;
    tObs.qThetaSum = 0;
}
tObs.qKomega = KOMEGAFILTED;
tObs.qOmegaFilted = QLowPassFilter(tObs.qOmegaFilted, tObs.qOmega, tObs.qKomega);

sTemp = tObs.qOmegaFilted;
if (sTemp < OMEGA300RPM)
    sTemp = OMEGA300RPM;
tObs.qKafc = tObs.qKafcFinal = MulFrac(sTemp, PI_DIV_IRP_PERCALC);
tObs.qTheta = qThetaTemp + EANGLE90;

Temp = (unsigned int)tObs.qOmegaFilted;
E_Frequency32.U32 = __builtin_muluu(Temp, 1000);
Temp32A.U32 = __builtin_muluu(E_Frequency32.Words.Low, 60);
Temp32B.U32 = __builtin_muluu(E_Frequency32.Words.High, 60);
tObs.Erpm = Temp32B.Words.Low + Temp32A.Words.High;
```

这样的话，整段程序，我们就分析清楚了。估算出角度，求角度变化量，然后滤波，求解自适应滤波系数，再求解转速。



本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

这个这幅框图，对应的代码，我们都分析完了。然后呢，我们也可以把顺时针，逆时针都兼容掉。

```
qThetaTemp = CalAtan(tObs.qEbetaFinal, tObs.qEalphaFinal);

if(tFoc.Direction == CCW)
    tObs.qThetaSum += qThetaTemp - tObs.qThetaLast;
else
    tObs.qThetaSum += tObs.qThetaLast - qThetaTemp;
```

一个是求角度差这个地方。

```
if(tFoc.Direction == CCW)
    tObs.qTheta = qThetaTemp + EANGLE90;
else
    tObs.qTheta = qThetaTemp - EANGLE90;
```

另一个是角度补偿这个地方。之前有人问，能否正反转自然切换，我们是无感算法，转速低的时候反电动势接近零是不能运行的，正反切换需要经过转速接近零的阶段，所以不好实现啊好的，那我们继续往下讲哈。

问：3 根电机电源线任意更换就能反转么？
答：他们指的是不更换接线的情况下。

```
void Observer(void)
{
    PUSHCORCON();
    CORCONbits.SATA = 1;
    CORCONbits.SATB = 1;
    CORCONbits.ACCSAT = 1;      //A B 40bit

    tObs.qUalpha = tUm.qUalphaOld;
    tObs.qUbeta = tUm.qUbetaOld;

    tObs.qIalpha = tIm.qIalpha;
    tObs.qIbeta = tIm.qIbeta;

    tObs.qIalphaEstimate = EstimateCurrent(tObs.qIalphaEstimate, tObs.qG, tObs.qUalpha, tObs.qEalpha, tObs.qZalpha, tObs.qF);
    tObs.qIbetaEstimate = EstimateCurrent(tObs.qIbetaEstimate, tObs.qG, tObs.qUbeta, tObs.qEbeta, tObs.qZbeta, tObs.qF);

    CalSlideModeZ();

    tObs.qEalpha = QLowPassFilter(tObs.qEalpha, tObs.qZalpha, tObs.qKafc);
    tObs.qEbeta = QLowPassFilter(tObs.qEbeta, tObs.qZbeta, tObs.qKafc);

    tObs.qEalphaFinal = QLowPassFilter(tObs.qEalphaFinal, tObs.qEalpha, tObs.qKafcFinal);
    tObs.qEbetaFinal = QLowPassFilter(tObs.qEbetaFinal, tObs.qEbeta, tObs.qKafcFinal);

    CalEangle();

    POPCORCON();
}
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

我们分析的电流观测器的代码函数，都放在这个 Observer 中调用。上面代码包含了 计算估算电流，计算控制器输出，对控制器输出低通滤波器，计算角度，角度补偿，求解转速，求解自适应滤波系数。这个函数的话，可以放到中断中调用。

```
//50us Interrupt
void __attribute__((section(".main_section"),interrupt,shadow,n
{
    IFS0bits.AD1IF = 0;

    AdcSample();
    GetCurrent();
    Observer();
    CurrentLoop();
    StatusControl();
    CalSinCos(tFoc.Eangle_Rotor);
    AdcFilter();
    SpiTransmit();
    DownCounter();
}
```

也就是 50us 执行一次这个函数，然后，去估算反电动势合成矢量角。

```
tObs.qIalphaEstimate = EstimateCurrent(tObs.qIalphaEstimate, tObs.qG, tObs.qUalpha, tObs.qEalpha, tObs.qZalpha, tObs.qF);
tObs.qIbetaEstimate = EstimateCurrent(tObs.qIbetaEstimate, tObs.qG, tObs.qUbeta, tObs.qEbeta, tObs.qZbeta, tObs.qF);
```

但是，实际上呢，我们这里还有一些系数没有确定出来。例如，这里的 F 跟 G，还有控制器的最大误差跟最大输出 K。 F 跟 G 的话，是通过测量电阻 R 跟电感量 L 来确定的。

```
void ObserverInitialize(void)
{
    tObs.qF = CONST_QF;
    tObs.qG = CONST_QG;

    tObs.qZErrMax = CONST_QZERRMAX;
    tObs.qZKSlide = CONST_QZKSLIDE;
}
```

这里呢，我们写了一个初始化函数，F 跟 G 的话，也是用宏定义定义出来的。

```
//Rs = 16.37R  Ls = 13.921mH
#define CONST_QF                30885    //1 - Ts * R / L = 1 - 50 us x 8 Ohm / 6960.5 uH = 0.94253286401838948351411536527548
#define CONST_QG                235      //Ts / L = 50 us / 6960.5 uH = 0.00718339199770131456073557934056
```

大家看下，之前我们使用电桥实测电阻跟电感量。测量出来，线电阻是 16.37R，线电感是 13.921mH。相电阻跟相电感的话，就是对这两个值除 2。根据公式，F = 1- Ts*R/L，

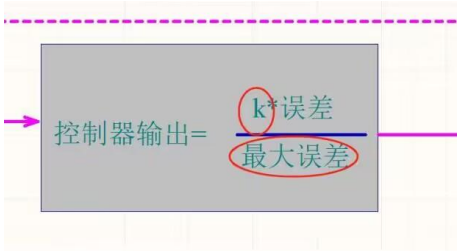
本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

G = Ts/L，就可以把这两个值计算出来。计算出来之后呢，再转换为 Q15 的值。也就分别对应 30885 跟 235。这是这两个参数的计算。

```
/*==========================================
void ObserverInitialize(void)
{
    tObs.qF = CONST_QF;
    tObs.qG = CONST_QG;

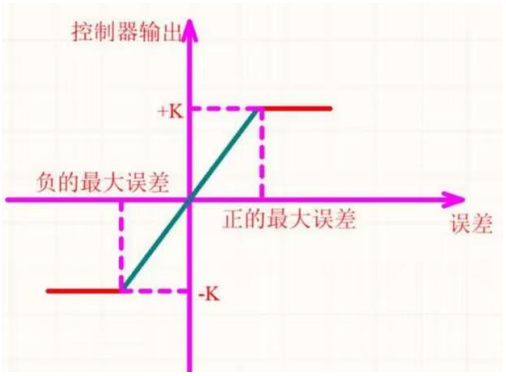
    tObs.qZErrMax = CONST_QZERRMAX;
    tObs.qZKSlide = CONST_QZKSLIDE;
}
```

那么，下面的重点呢，是要把控制器的最大误差跟最大输出 K 确定出来。



也就是这两个参数。之前，我们讲过，要想电流观测器计算出来的估算电流能够很好的收敛到实际电流的话，就需要对控制器相关的参数进行设置。也就是说，这两个值不能随便给，而是要调试确定。

只有设置合适的值，才能确保估算电流能够很好的跟踪上实际电流。所以啊，下面我们就重点来介绍一下控制器中，两个重要参数的整定方法。那么，这两个参数的作用，我们稍微回顾一下啊。最大误差这个参数，它是用来决定线性区的大小的。



如这幅图所示，我们之前分析过，如果线性区越大的话，能够改善由开关作用带来的振荡情况。反之如果线性区越小，开关作用会越明显，振荡情况就会变得更加严重。但是呢，线性区设置过大，虽然能够改善振荡情况，可以它会导致估算电流的收敛速度变慢。所以呢，基于上面原因，最大误差这个参数，设置不宜过大。这个参数带来的影响，我们会在后面分析波形的时候，大家会看到。然后呢，另一个参数是最大输出 K。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

如果说最大输出 K 设置越大，收敛的速度会更快，但是太大的话，也会引起震荡。但是，这个值太小的话，会导致估算电流始终跟踪不上实际电流。所以说，这个值，需要在不震荡的情况下，让估算电流跟踪上实际电流。下面呢，就以一些实际的波形来给大家分析一下，两个参数的影响。

```
#define CONST_QZERRMAX                  0       //
#define CONST_QZKSLIDE                  0   //
/*=================================版权归张飞实战电子所有，违者必究
```

因为我们要调整这两个参数，所以呢，初始值可以写 0，然后开始调整。调整的时候呢，先设置一个初值。刚开始的时候，我们可以把最大误差设置得小一些，线性区窄一些，最大输出 K 也设置小一些。 设置最大误差为 0.1%，也即是 0.1%*32768 = 32.768 = 32，增益 K 这个参数，也一样设置为 0.1%，对应的也是 32。

```
#define CONST_QZERRMAX                  32      //
#define CONST_QZKSLIDE                  32  //
```

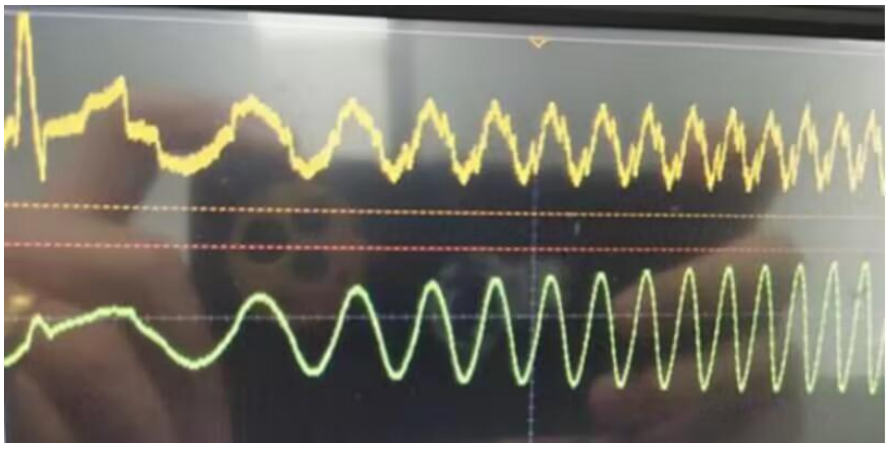
此时呢，我们使用 SPI 把估算电流跟实测电流送到 DAC 芯片，然后使用示波器观察波形。

```
static void SpiDataUpdate(void)
{
    unsigned int Temp;

    Temp = (unsigned int)(tObs.qIalphaEstimate + 0x8000);
    tSpi.ReadyData[0] =  Temp >> 4;

    Temp = (unsigned int)(tIm.qIalpha + 0x8000);
    tSpi.ReadyData[1] =  Temp >> 4;
}
```

这样的话，大家找到 DAC 芯片输出引脚，测量对应的通道波形。

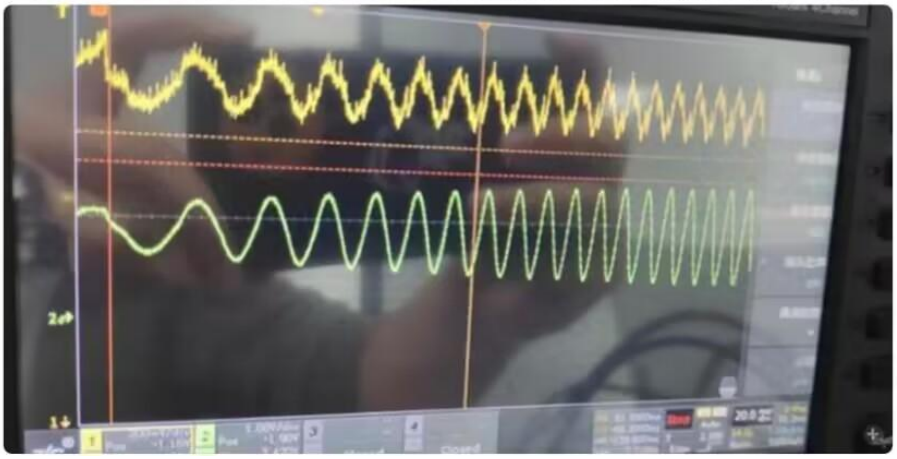


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

大家看下，这是之前实测的一个波形。黄色的是实际测量的电流波形，绿色的是估算的电流波形。很明显，估算的电流此时是没有跟踪上实际电流的。那么，很显然呢，此时的参数就是不合理的。不合理的话，就要调整参数。

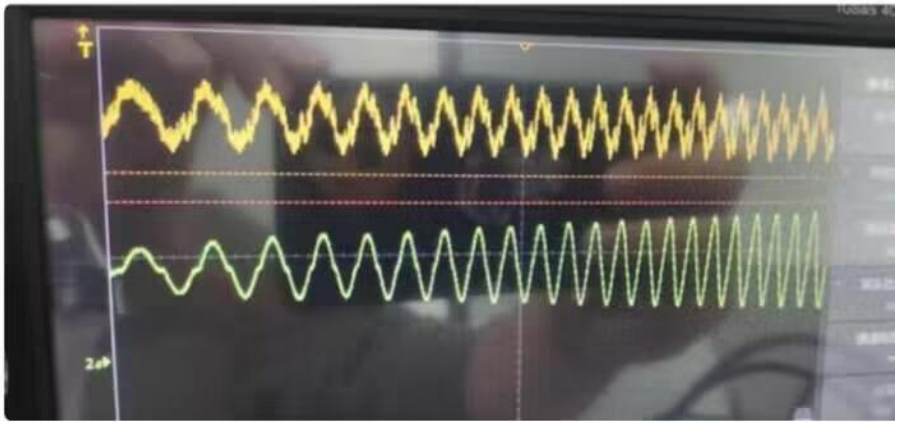
调整的策略呢，我们可以先保持一个参数不变，来修改另一个参数。例如，我们可以保持最大误差的设置不变，修改增益 k， 让增益从 0.1%变为 1%，看看效果。

```
#define CONST_QZERRMAX                  32      //
#define CONST_QZKSLIDE                  327   //
/*=============================版权归张飞实战电子所有  请勿
```



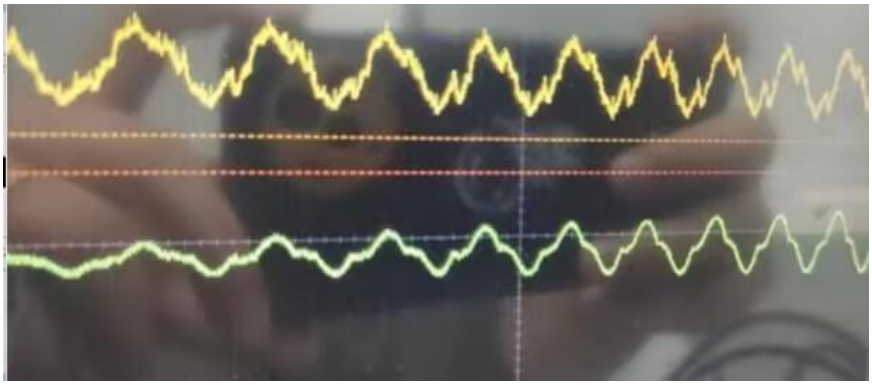
此时，再看实测波形，似乎跟之前的没什么变化。没有变化的话，我们继续加大最大输出 K,例如，我们可以变为 10%。

```
#define CONST_QZERRMAX                  32      //
#define CONST_QZKSLIDE                  3277   //
```

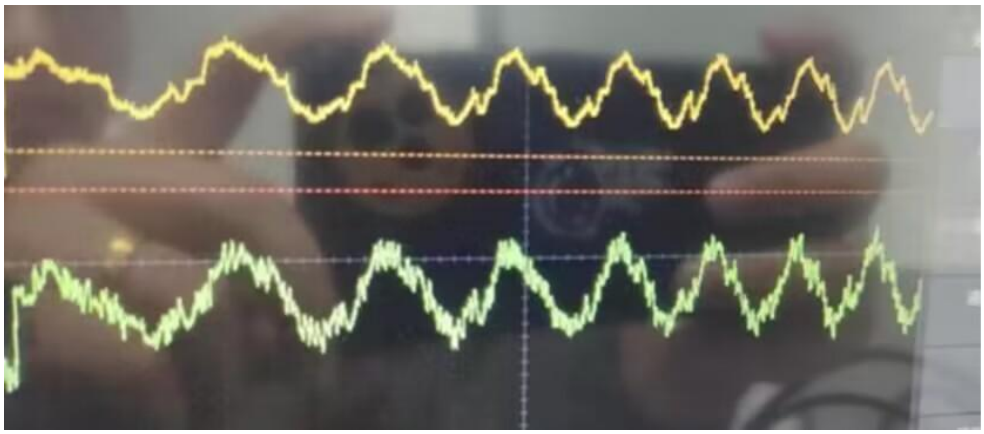


这个时候，大家可以看到，估算电流有了一定变化。
但是呢，效果还是不好，然后，继续去增加最大输出 K。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



这是变为 20%的情况，按照这个方法去调整，不断去加大。



随着 K 的加大，我们会看到这样的波形。

```
#define CONST_QZERRMAX                  32      //
#define CONST_QZKSLIDE                  16384   //
```

此时对应的 K 是这样的。

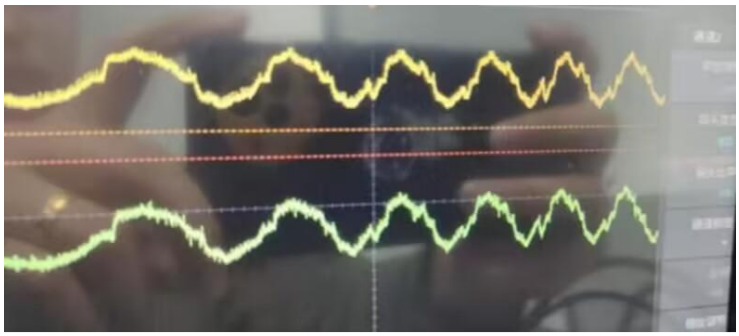
问：实际电流怎么不够正弦？
答：和电机本身设计有关，BLDC 本身正弦就不是很好。电流波形取决于 控制器 + 电机。

大家会发现，绿色波形已经跟踪上黄色波形了，但是呢，虽然跟踪上了，波形中的高频成分也比较多。而这个高频成分，是跟最大误差设置有关的。最大误差设置比较小的话，线性区比较窄，可能就会引入震荡。此时呢，我们可以尝试把最大误差加大一些。

```
#define CONST_QZERRMAX                  327     //
#define CONST_QZKSLIDE                  16384   //
```

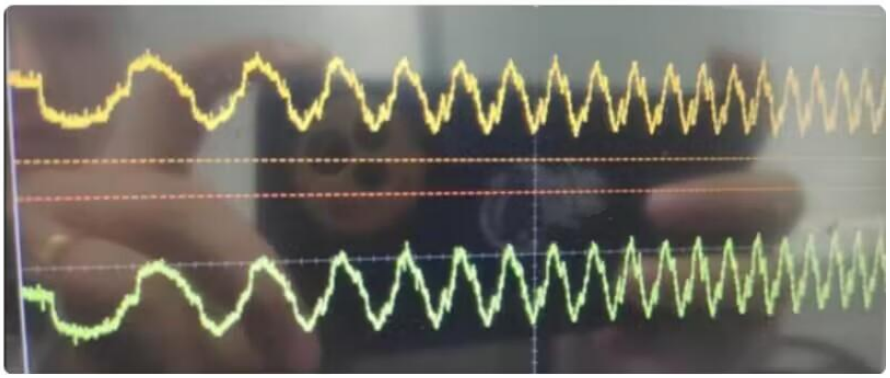
例如，变为这样。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

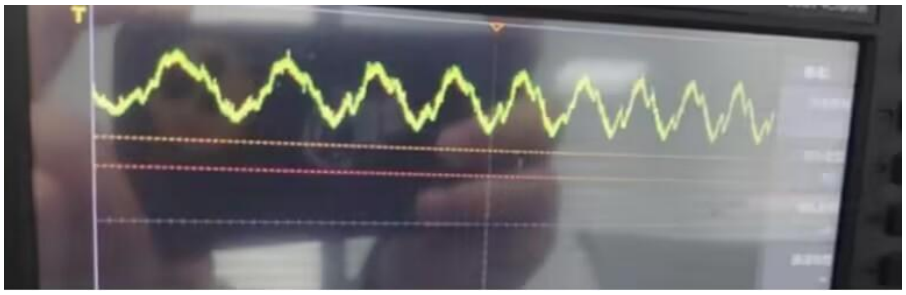


此时，看下波形，很明显，绿色波形中的高频成分减少了。如果觉得高频成分还比较大，我们可以尝试把最大误差再加大一些。

```
#define CONST_QZERRMAX          512     //
#define CONST_QZKSLIDE          16384   //
```



仔细对比看下这个波形，波形变得更好了。此时我们可以看到，估算电流已经基本收敛到实际电流了。



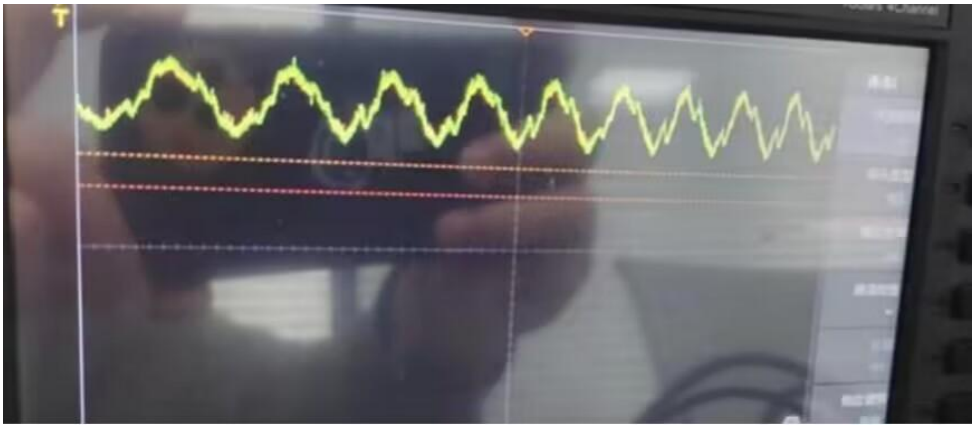
我们把两个波形都放在一起，基本两个波形重合了。

问：估算的是 alfa 电流 实际如何测这个 alfa 电流？
答：用 DAC 芯片输出变量观测，alfa 电流是输入电流经 clark 变换后的电流。

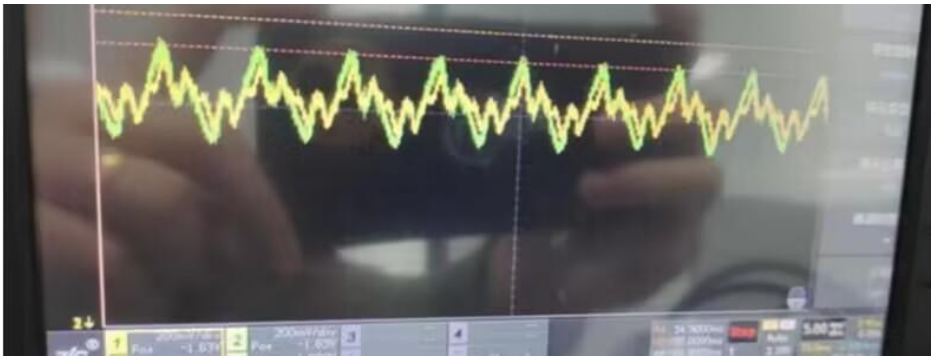
问：如果电机是无感电机呢一般建议怎么操作呢？
答：研发的时候装传感器，批量生产的时候没有传感器。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

让绿色跟踪上黄色波形，就是我们的目标。所以说，我们调试的时候，必须要看着波形来调整的。



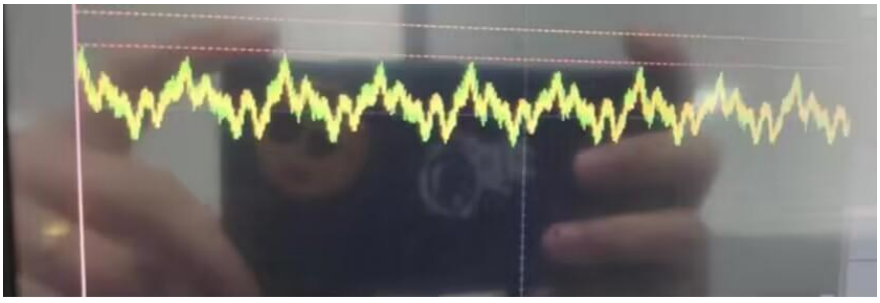
那么，大家把波形调整到这样之后，要再重复启动再测试看看。



如果碰到这样波形，跟踪不是很好。这样就需要再调整下参数，可以把最大输出 K 再调整大一些。

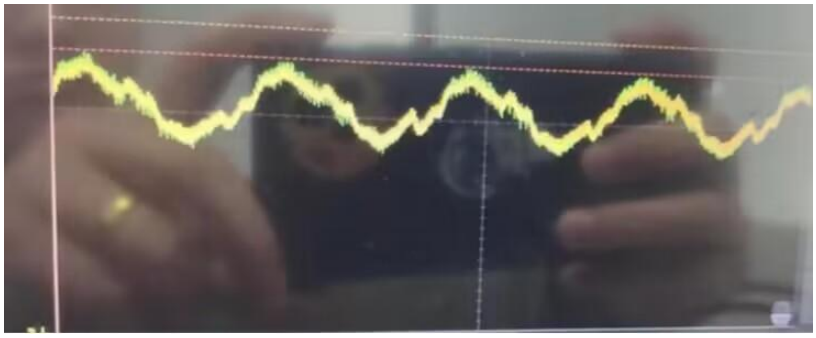
```
#define CONST_QZERRMAX                512     //
#define CONST_QZKSLIDE                19660   //
/*=======================================版权归张飞实战电子所有.
```

例如，由 16384，变为 19660。



本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

此时，对应波形跟踪变好了，然后，再加点载看看。



这是加载之后的波形，可以看到，跟踪效果还是可以的。

好，那我们今天就先讲到这里哈。大家有条件，可以自己下去调试看看。既然这个观测器调试好了，我们后面就要抛弃掉传感器了。要使用无感的方式来控制电机运转。

大家晚安！

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。